SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q4 – Two-Dimensional Arrays: District Rainfall Records

```
#define D 38
#define M 12
double rain[D][M];                    /* declared; read with nested loops */
void row_totals(int d,int m,double r[][M],double tot[])   /* annual total per district */
{ for(int i=0;i<d;i++){ tot[i]=0; for(int j=0;j<m;j++) tot[i]+=r[i][j]; } }
void col_avg(int d,int m,double r[][M],double avg[])      /* state monthly average */
{ for(int j=0;j<m;j++){ double s=0; for(int i=0;i<d;i++) s+=r[i][j]; avg[j]=s/d; } }
void max_elem(int d,int m,double r[][M],int *bi,int *bj)  /* wettest district-month */
{ *bi=*bj=0; for(int i=0;i<d;i++) for(int j=0;j<m;j++) if(r[i][j]>r[*bi][*bj]){*bi=i;*bj=j;} }
int deficient(int d,const double tot[],int out[])         /* below 60% of state average */
{ double sum=0; for(int i=0;i<d;i++) sum+=tot[i]; double mean=sum/d;
  int c=0; for(int i=0;i<d;i++) if(tot[i] < 0.6*mean) out[c++]=i; return c; }
```

(Comparing a district's annual total with 60% of the mean annual total is the same as comparing its monthly average with 60% of the state monthly average.)

Student – Array Design and Memory

- **Row-major order:** C stores rain[38][12] as 38 rows one after the other: rain[0][0] … rain[0][11], rain[1][0] … – 456 doubles in one contiguous block of 3,648 bytes (measured sizeof).
- **Address formula:** &rain[i][j] = base + (i × 12 + j) × sizeof(double). Check: &rain[2][3] − &rain[0][0] = 27 elements = 2 × 12 + 3 (measured).
- **Loop order:** the inner loop over j visits consecutive addresses, so each 64-byte cache line (8 doubles) is used fully and the hardware prefetcher can run ahead. Looping over i in the inner loop jumps by a whole row (12 × 8 = 96 bytes here; 16,000 bytes for a 2,000-column array), so almost every access touches a new cache line. For a large array this causes many cache misses and a slower traversal.
- **3-D extension:** double rain[YEARS][38][12]; address = base + ((y × 38 + d) × 12 + m) × sizeof(double). The last index changes fastest, so the innermost loop should be over months.

Student – Implementation and Verification

Hand-checked 3 × 4 array: rows {10, 20, 30, 40}, {5, 5, 5, 5}, {100, 0, 50, 25}.

Quantity	By hand	Program output
Row totals	100, 20, 175	100 20 175
Column averages	(10+5+100)/3 = 38.33; 8.33; 28.33; 23.33	38.33 8.33 28.33 23.33

Quantity	By hand	Program output
Maximum	100 at row 2, column 0	100 at [2][0]
Deficient (below 60% of mean total 98.33 = 59)	row 1 (total 20)	row 1

Traversal of a 2,000 × 2,000 array of doubles (32 MB), compiled with gcc -O0, two runs: row-wise 12.30 ms and 11.69 ms; column-wise 15.62 ms and 16.39 ms – column-wise is about 1.3–1.4× slower.

Explanation: both loops do the same 4 million additions, but the column-wise loop touches a different cache line on almost every access (a stride of 16,000 bytes), so far more accesses come from main memory. The ratio here is modest because the hardware prefetcher recognises the constant stride and the work per element is small; it is larger on machines with smaller caches or for larger arrays. (With -O2 the compiler may vectorise or remove the row-wise loop entirely, so measure at -O0 or use a volatile accumulator for a fair comparison.)

Joint conclusion – why 2-D rather than 38 separate arrays: all districts are processed by the same code in loops over i and j, instead of 38 copies of that code; a function takes one parameter (the table) instead of 38; the data stays in one contiguous block (better cache use); totals, averages and maxima across districts (columns) are just loops, whereas separate arrays cannot be indexed by district number.

Alternative Concept and Complexity Analysis

Representation	Access time	Extra space	Layout / speed
rain[38][12]	O(1): one multiply, one add	none	one contiguous block; compiler does the index arithmetic
1-D array, index i × 12 + j	O(1): same arithmetic written by hand	none	identical memory layout and speed; the programmer must keep the 12 consistent
Array of 38 pointers, each row malloc'd	O(1): load pointer, then index	38 × 8 = 304 bytes of pointers + allocator overhead per row	rows are contiguous inside, but not between each other; one extra load per access; allows rows of different lengths and resizing

In all three, row-wise traversal is fast and column-wise traversal is slow, because the speed difference comes from memory layout (stride), not from the declaration style. The pointer version can be slightly slower from the extra pointer load and the loss of contiguity.